Université Lumière Lyon 2
Master Informatique

2026 – 2027

APPRENTISSAGE SUPERVISÉ DE RÉSEAU DE NEURONES

TD01 – Le Perceptron

Tous les travaux de cette matière sont publiés via : Lien du dépôt

RÉALISÉ PAR :
Evan DYNAK

1 Réponses aux questions

1.1 Perceptron et problème jouet

1.1.1 Droites de séparation obtenues et capacités en généralisation

Pour les fonctions booléennes linéairement séparables (OU et ET), l'algorithme du Perceptron s'arrête dès que tous les exemples d'entraînement sont parfaitement classés ($n_{erreurs} = 0$). L'analyse géométrique et numérique de ces solutions met en évidence plusieurs propriétés fondamentales :

1. **Non-unicité de la droite de séparation :** Pour un jeu de données linéairement séparable, il existe une infinité de droites séparatrices valides. L'algorithme s'arrête à la première droite sans erreur rencontrée. L'hyperplan final dépend donc entièrement du vecteur de poids initial $W_{initial} = (w_1, w_2, w_3)$ tiré aléatoirement et de l'ordre de parcours des exemples.
2. **Absence d'optimisation :** Le Perceptron ne possède aucun critère de marge maximale. La frontière de décision finale peut passer arbitrairement près de certains points d'apprentissage.

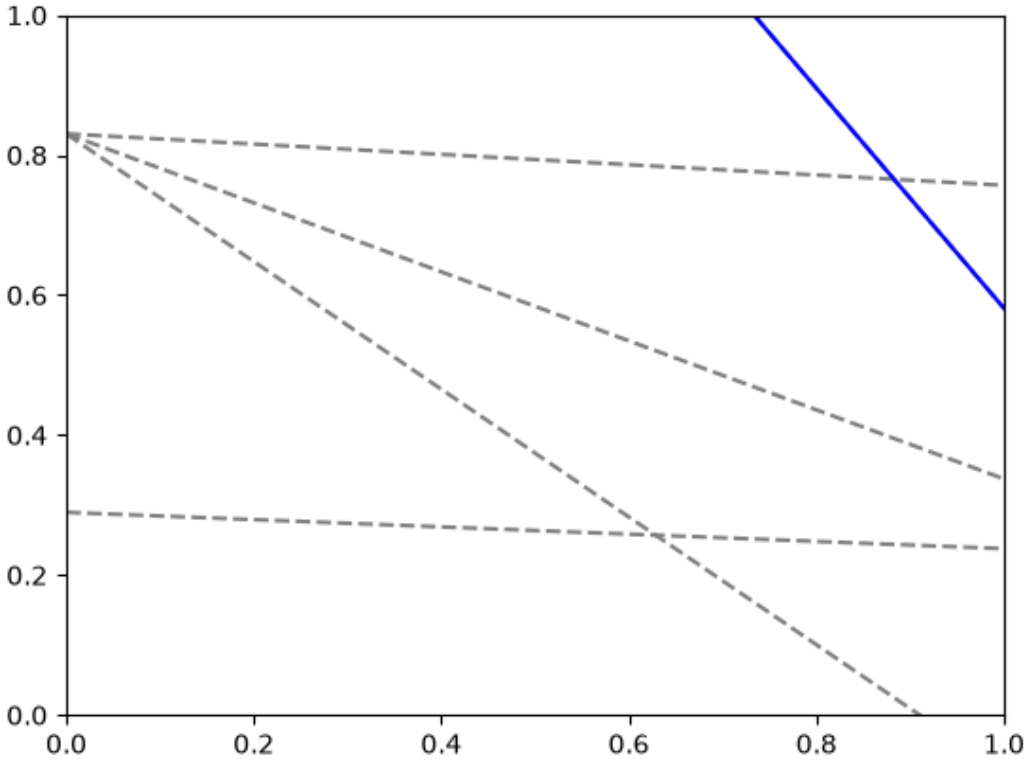


Figure 1 Rendu final pour l'opérateur ET

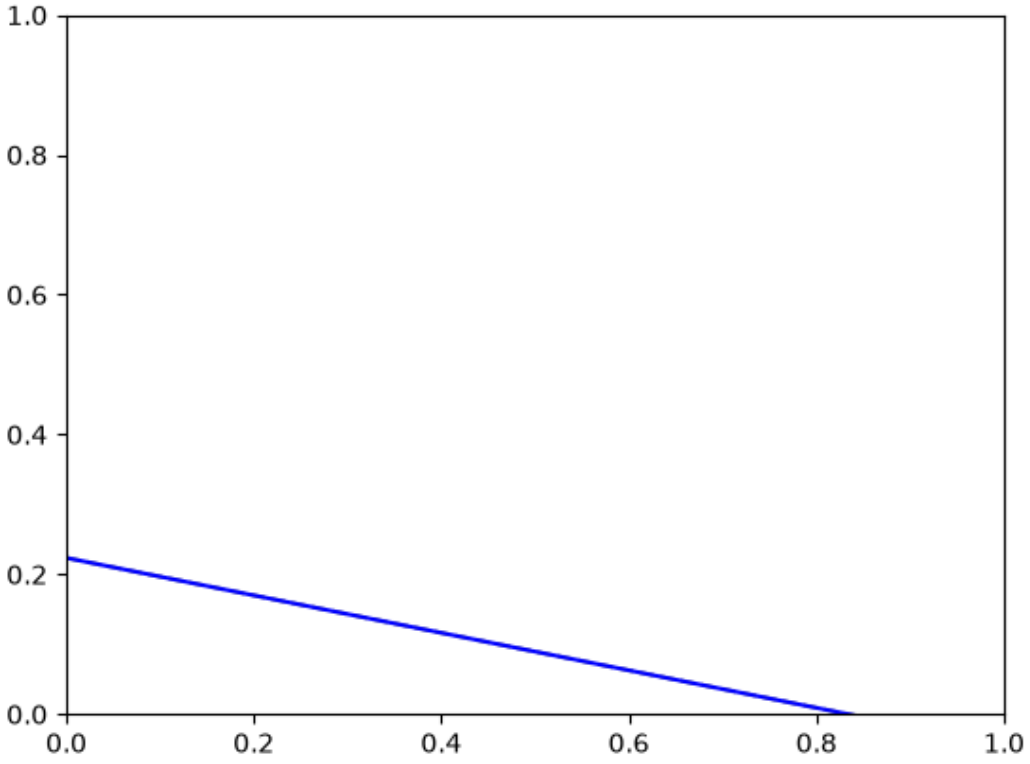


Figure 2 Rendu final pour l'opérateur OU

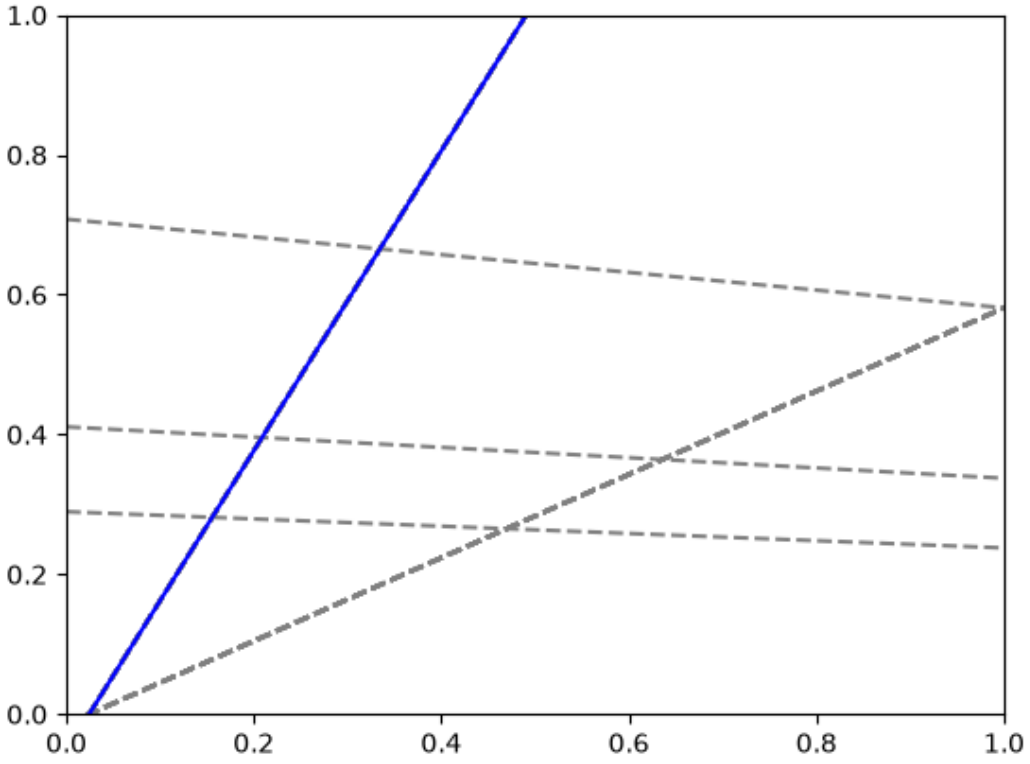


Figure 3 Rendu final pour l'opérateur XOR

1.1.2 Difficultés face au OU exclusif (XOR)

- **Comportement observé :** Pour la fonction XOR, l'algorithme du Perceptron est incapable de converger ($n_e \geq 2$ à chaque époque). Les poids oscillent indéfiniment sans jamais annuler l'erreur. L'exécution s'interrompt uniquement par le critère d'arrêt sur le nombre maximal d'itérations fixé ($NB_ITERATIONS = 20$).
- **Justification théorique :** Le Perceptron à un seul neurone calcule une combinaison linéaire $w_1x_1 + w_2x_2 + w_3$ suivie de la fonction seuil. Sa frontière de décision est par conséquent un hyperplan affine. Or, la fonction XOR n'est **pas linéairement séparable** . Aucune droite ne peut donc isoler $(1, 0), (0, 1)$.
- **Solution :** Utiliser un Perceptron multi-couche (minimum 2 : $\text{XOR}(x_1, x_2) = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$ avec un neurone de sortie par exemple) permettant de décomposer ce problème non linéaire en deux sous problèmes linéaires.

1.2 Problème réel en grande dimension

1.2.1 Que faire si le Perceptron a des difficultés à converger ?

Dans le cas de données réelles à grande échelle, des difficultés de convergence se traduisent par de lentes oscillations, une instabilité de la trajectoire des poids ou une stagnation prolongée :

1. **Normalisation / Standardisation des données :** Centrer et réduire les données ou normaliser dans l'intervalle $[0, 1]$ ou $[-1, 1]$. Lorsque certaines coordonnées possèdent des échelles de grandeur disparates, les lignes de niveau de l'erreur forment des vallées très étirées (mauvais conditionnement), provoquant d'importants dépassements orthogonaux à chaque mise à jour.
2. **Ajustement et décroissance du pas d'apprentissage :** Un pas excessif entraîne des oscillations permanentes autour de l'hyperplan optimal. Il convient de réduire le pas.
3. **Mélange aléatoire des exemples :** Mélanger l'ordre des exemples à chaque époque afin de briser d'éventuels cycles de mise à jour périodiques qui enferment les poids dans des boucles infinies.

1.2.2 Que faire si le Perceptron n'arrive pas du tout à converger ?

Si le Perceptron ne parvient aucunement à converger, cela signifie que la base d'exemples n'est **pas linéairement séparable**.

1. **Adopter un modèle non linéaire :** Déployer un réseau de neurones multicouche doté de fonctions d'activation non linéaires entraîné par descente de gradient, ou séparé le problème non linéaires en plusieurs sous problèmes linéaires et les appliquer en multicouche.
2. **Utiliser les méthodes à noyaux :** Projeter implicitement les vecteurs d'entrée dans un espace de caractéristiques de dimension très élevée où les données redeviennent séparables sans coût calculatoire excessif.
3. **Nettoyage des données et réduction de dimensionnalité :** Éliminer les points aberrants, rectifier les erreurs d'annotation pour supprimer le bruit non pertinent.

2 Code Source Final

Le script Python complet ci-après met en œuvre l'ensemble des fonctionnalités requises par le sujet de TD : initialisation aléatoire, apprentissage vectorisé, affichage en temps réel de l'évolution de l'erreur, calcul explicite des équations de séparation et génération automatique des figures comparatives sous `Matplotlib`.

```python
import matplotlib.pyplot as plt
from random import random

f_activation = lambda x: 0 if x < 0 else 1
W_initial = [random(), random(), random()]

OU = [  ([0,0,1], 0),
        ([0,1,1], 1),
        ([1,0,1], 1),
        ([1,1,1], 1) ]

XOR = [  ([0,0,1], 0),
        ([0,1,1], 1),
        ([1,0,1], 1),
        ([1,1,1], 0) ]

ET = [  ([0,0,1], 0),
        ([0,1,1], 0),
        ([1,0,1], 0),
        ([1,1,1], 1) ]

def demarrer(exemples):
    W = W_initial.copy()
    eta = 0.2
    ne = 1
    NB_ITERATIONS = 20
    iterations = 0

    plt.figure()
    plt.xlim([0,1])
    plt.ylim([0,1])

    while ne != 0 and iterations < NB_ITERATIONS:
        ne = 0
        for index in range(len(exemples)):
            X,d = exemples[index]
            S = f_activation(W[0]*X[0] + W[1]*X[1] + W[2]*X[2])
            E = d - S
            if E != 0:
                ne += 1
                for i in range(len(W)):
                    W[i] += eta * E * X[i]
                a, b = -W[0] / W[1], -W[2] / W[1]
                y = [b, a * 1 + b]
                plt.plot([0, 1], y, color='gray', linestyle='--')
        iterations += 1
        print(ne)

    if ne > 0:
        print("L'opération ne converge pas.")

    a, b = -W[0] / W[1], -W[2] / W[1]
    plt.plot([0, 1], [b, a + b], color='blue')
    plt.show()
```

```
55
56 print("----- OU -----")
57 demarrer(OU)
58 print("----- ET -----")
59 demarrer(ET)
60 print("----- XOR -----")
61 demarrer(XOR)
```

Listing 1 Script complet `td01.py` : Apprentissage du Perceptron pour les fonctions booléennes OU, ET et XOR